

前言

第18章开发的 **多包仓库** 的相关模块陆续发布到 **Npm公有仓库** 与 **Npm私有仓库**，其他开发者可通过**Npm公有仓库**或**Npm私有仓库**找到相关模块查看 **Readme** 了解其信息。

经常使用 **Npm** 的同学可能会发现以下细节。很多明星项目的 **Readme** 很简单，甚至不愿多说一句话。像 **react** 与 **babel**，每周下载量达到上千万，就连一个简单的 **Readme** 都不写好，那是什么意思啊！

The screenshot shows the npm package page for 'react'. The page is titled 'react' and shows version '18.0.0'. It includes a 'Readme' tab, an 'Explore' tab, and a 'BETA' label. The package has 1 dependency, 85,523 dependents, and 874 versions. The 'Install' section shows the command 'npm i react'. The 'Repository' section points to 'github.com/facebook/react'. The 'Homepage' section points to 'reactjs.org/'. The 'Weekly Downloads' section shows 14,097,943 downloads. The 'Version' section shows '18.0.0' and the 'License' section shows 'MIT'. The 'Unpacked Size' section shows '316 kB' and the 'Total Files' section shows '20'. The 'Issues' section shows '675' and the 'Pull Requests' section shows '726'. The 'Keywords' section is empty.

react 18.0.0 • Public • Published a month ago

Readme Explore BETA 1 Dependency 85,523 Dependents 874 Versions

react

React is a JavaScript library for creating user interfaces.

The `react` package contains only the functionality necessary to define React components. It is typically used together with a React renderer like `react-dom` for the web, or `react-native` for the native environments.

Note: by default, React will be in development mode. The development version includes extra warnings about common mistakes, whereas the production version includes extra performance optimizations and strips all error messages. Don't forget to use the **production build** when deploying your application.

Example Usage

```
var React = require('react');
```

Keywords

Install

```
> npm i react
```

Repository

github.com/facebook/react

Homepage

reactjs.org/

Weekly Downloads

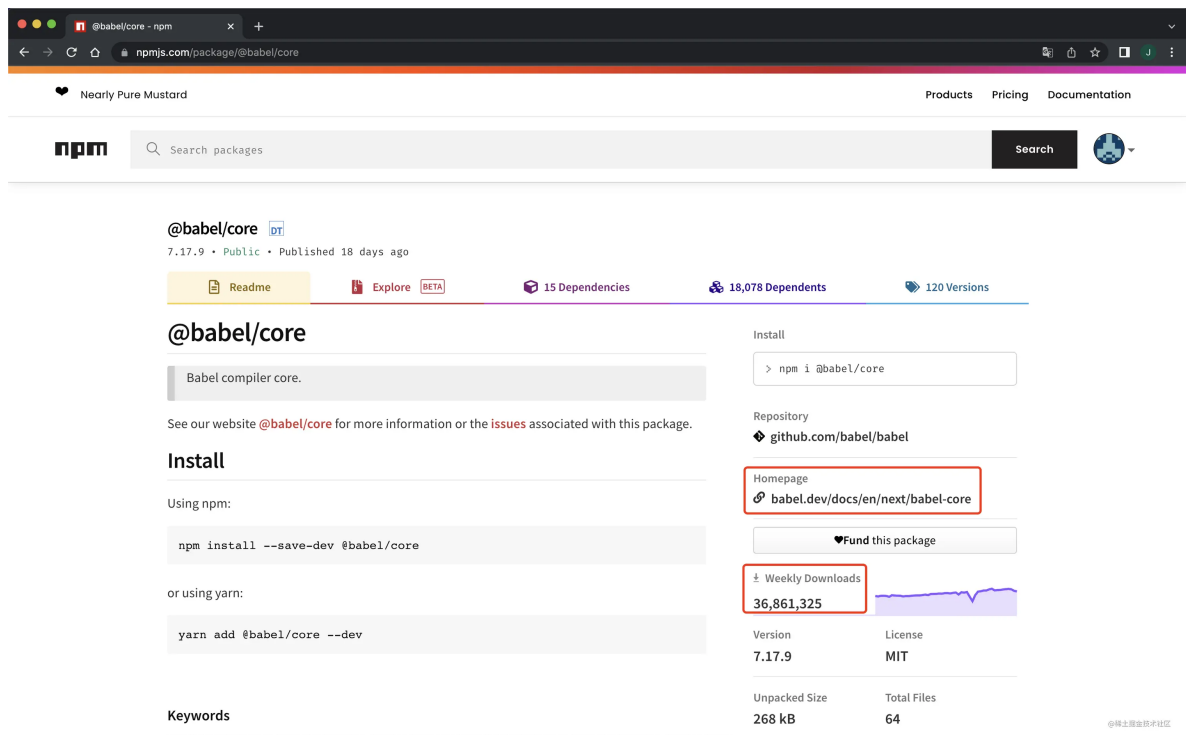
14,097,943

Version	License
18.0.0	MIT

Unpacked Size	Total Files
316 kB	20

Issues	Pull Requests
675	726

@稀土掘金技术社区



其实并不然，因为它们整个项目的量级巨大且面向全球开发者，一份标准规范的 **Readme** 难以承受关于项目的全部说明，因此需使用一个 **文档站点** 作为载体，为开发者提供该项目更多的说明。在这种情况下，这些明星项目的 **Readme** 通常都很简单，有些简单到甚至整个篇幅只有一句话，而它们的文档已全部转移到 **文档站点** 中了。

仔细看看 **Npm官网** 每个模块右边栏目中都有一个 **Homepage** 信息，该信息可设置一个官网地址或文档地址，点击就会直接跳转到该模块的官网站点或 **文档站点** 了。配置 **Homepage** 也很简单，在 **package.json** 中指定 **homepage**，当模块发布到 **Npm公有仓库** 就会立即生效。

```
{  
  "homepage": "https://doc.yangzw.vip/bruce"  
}
```

一个良好的项目文档使用 **Readme** 还是 **文档站点**，还得根据情况来定。

从 **Readme** 的角度来看，适合小型项目。因为小型项目的规模不大，功能不多，用户量级也不大，一份 **Readme** 就能将背景、安装、使用、徽章、示例、维护、贡献、证书等信息展示完毕，其他开发者大概浏览就能得知该项目的重点信息。

从 **文档站点** 的角度来看，适合大型项目或开源项目。因为这些项目的规模较大，功能较多，版本较多、迭代速度较快，用户量级也很大，一个 **文档站点** 才能充分展示更多信息，另外像路由页面、国际语言、代码演示、在线演示、实时编辑等动态功

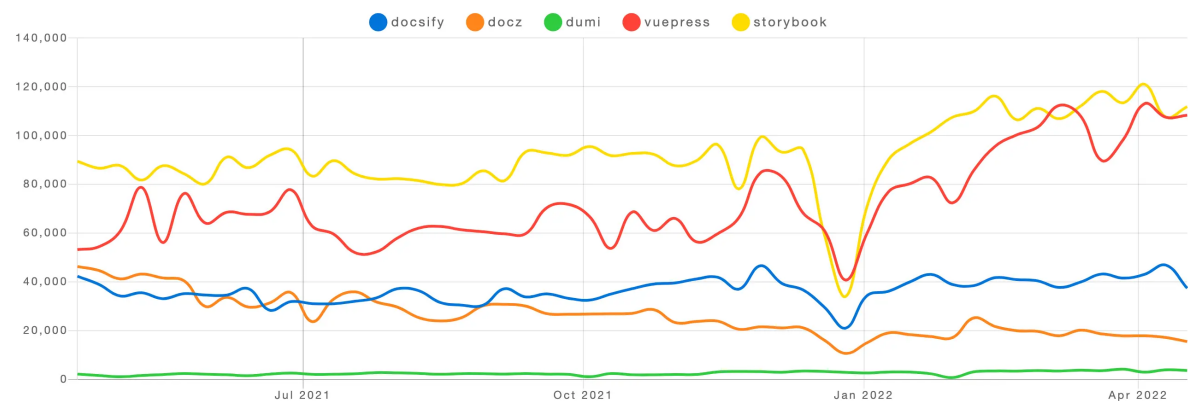
能也只有 文档站点 才能支持，而 Readme 充其量就是一个 静态HTML ，压根不能实现这些功能。

正是这样，很多明星项目的 Readme 很简单但 文档站点 却很丰富，一份完善且漂亮美观的文档更易吸引其他开发者，就像大多数男人都喜欢看美女一样。为了能让自己开发的项目也一样吸引人，门面功夫还得做好，而搭建一个漂亮美观的 文档站点 ，是势在必行的。本章将带领你部署一个属于自己的技术文档站点，完善文档的必要细节再部署到自己的域名中，让更多开发者叹为观止。

背景：文档生成是什么

目前主流的 前端文档生成框架 主要有 docsify 、 docz 、 dumi 、 vuepress 与 storybook 。 storybook 在国外很受欢迎，喜欢 vue 技术栈的同学会发现与 vue 相关项目几乎都使用 vuepress 生成 文档站点 。

Downloads in past 1 Year ▾



Stats

			Stars	Issues	Version	Updated ⑦	Created ⑦	Size
docsify			19,415	191	4.12.2	4 months ago	5 years ago	minzipped size 36.9 KB
docz			22,211	107	2.4.0	2 months ago	4 years ago	bundlephobia 429
dumi			2,169	70	1.1.40	a month ago	2 years ago	bundlephobia 429
vuepress			20,200	538	1.9.7	3 months ago	4 years ago	bundlephobia 429
storybook			70,473	1,810	6.4.22	10 days ago	6 years ago	bundlephobia 429 稀土掘金技术社区

你注意到图表底下那条绿色的曲线吗？它顽强且艰难地前进着，它就是 umi 生态中的 dumi 了，一个支持使用 react + typescript 搭建 文档站点 的工具。

对于这么多工具，该如何做选择？一个对比图标直观看看它们的区别。

工具	star	支持语言	基础功能	自
docsify	1.9w+	不限	✓	
docz	2.2w+	不限	✓	
dumi	0.2w+	react	✓	
vuepress	2w+	vue	✓	
storybook	7w+	不限	✓	

基础功能包括 [Readme](#) 一切编写语法，外加路由页面、国际语言、代码演示等强化功能，这些每个工具都支持，像自动生成 [API](#)、[导入模块到代码示例](#) 和 [文档内嵌到组件目录](#) 这样的动态生成功能，[dumi](#) 居然实现了，很难想象一个国产项目居然这么贴心，真是不可思议。

虽然 [dumi](#) 受欢迎程度比其他工具低，但使用下来给我的感觉是很贴心，能省去很多重重复复的工作。可能处于国际环境，[dumi](#) 并不受国外开发者的认可吧，就像国内开发者更多喜欢 [vue](#)，而国外开发者更多喜欢 [react](#) 一样。

[dumi](#) 的文档很完善，能带来很多惊喜，唯一的缺点就是不支持 [vue](#) 搭建 [文档站点](#)，但相信更多开发者使用起来，官方就会考虑兼容 [vue](#) 了。为了支持国产，因此首选 [dumi](#)。

方案：部署一个属于自己的技术文档站点

[dumi](#) 官网就是使用自身生成 [文档站点](#)，若配置移动端主题样式，还可在移动端设备中浏览，做到电脑手机平板多端浏览两不误。接着也是用 [dumi](#) 为第18章开发的 [多包仓库](#) 生成一个 [文档站点](#)。通过一步一步上手 [dumi](#)，解锁更多的 [dumi](#) 技能。



安装

因为 **文档站点** 在开发环境通过 **dumi** 提供的命令生成，所以将 **dumi** 安装到 **devDependencies**，为了兼容移动端，还需安装 **dumi-theme-mobile**。

```
yarn add dumi dumi-theme-mobile -D
```

安装完毕，在 **package.json** 中指定 **scripts**，加入生成命令。**deploy** 用于生成 **文档站点** 的静态文件，**docs** 用于调试 **文档站点**。

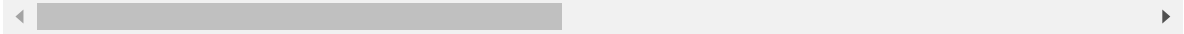
```
{
  "scripts": {
    "deploy": "dumi build",
    "docs": "dumi dev"
  }
}
```

配置

打开第18章开发的 **多包仓库**，在根目录中创建 **docs** 文件夹，在 **docs** 文件夹中创建 **index.md** 文件，加入以下内容。**dumi** 默认以 **docs** 文件夹的 **index.md** 为入口，遍历 **docs** 文件夹所有 **md** 文件，生成与目录结构一样的网页路径。

Bruce FEES

Bruce FEES是一套多功能**前端工程化多包管理**实践方案，包括多个前端开发流程中常见`脚手架`、



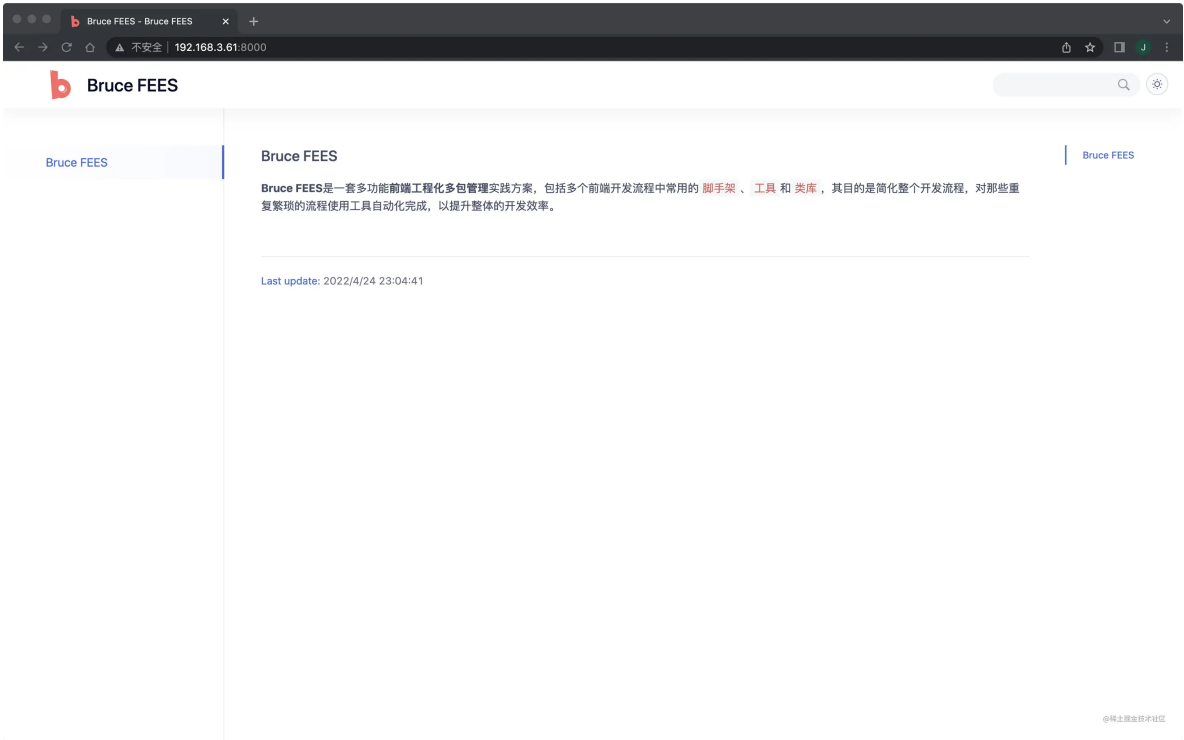
在根目录中创建 `.umirc.ts` 文件，加入以下内容。更多配置可查看[dumi配置列表](#)。

```
import { defineConfig } from "dumi";

const logo = "https://static.yangzw.vip/doc/logo.png";

export default defineConfig({
  favicon: logo,
  logo,
  mode: "site",
  title: "Bruce FEES"
});
```

一个基础配置文件就这么简单，执行 `yarn run docs` 启动应用。



接着将完善 **文档站点** 相关配置。像这种配置型工具，其界面与功能都已高度定制化，可修改官网文档提供的参数达到自己想要的效果，因此使用 `dumi` 可很快搭建一个 **文档站点**。虽然配置高度定制化，为了提升文档阅读体验，基于上述配置文件从以下方面逐步完善。

txt

1

在 `umirc.ts` 中指定 `hash`。当打包 文档站点 的静态文件时会哈希处理文件名称。再

†s

网页前缀

将 文档站点 的静态文件部署到服务器后，可通过 `https://yangzw.vip` 的域名访问 文档站点。考虑到后续可能还会部署更多 文档站点，推荐使用网页前缀区分，这样就不会占用主域名了。

例如将当前 多包仓库 的 文档站点 的路径定义为 `https://doc.yangzw.vip/bruce`，那以后其他 文档站点 的路径定义可定义为 `https://doc.yangzw.vip/xyz`，这样就能把 `https://doc.yangzw.vip` 保留下来干点别的了。

在 `.umirc.ts` 中指定 `base` 与 `publicPath`，让加载静态文件的路径增加前缀 `/bruce/`。

```
export default defineConfig({
  // ...
  base: "/bruce",
  publicPath: "/bruce/"
});
```

ts

文档导航

上述规划好目录结构，`dumi` 虽然能动态生成对应网页路径，但在站点中不存在任何入口。

 Bruce FEES



bruce-app

bruce-pkg

bruce-us

Github

个人官网



会稽山酒业有限公司

在 `.umirc.ts` 中指定 `navs`，显式设置文档导航的内容，可查看[navs配置](#)。

```
export default defineConfig({
  // ...
  navs: [
    { path: "/app", title: "bruce-app" },
    { path: "/pkg", title: "bruce-pkg" },
    { path: "/us", title: "bruce-us" },
    { path: "https://github.com/JowayYoung/bruce", title: "Github" },
    {
      children: [
        { path: "https://yangzw.vip", title: "🏠 首页" },

```

ts


```

    { path: "https://yangzw.vip/blog", title: "📖 博客" },
    { path: "https://yangzw.vip/source", title: "💻 开源" },
    { path: "https://yangzw.vip/tool", title: "🔧 工具" },
    { path: "https://yangzw.vip/store", title: "📦 收藏" },
    { path: "https://yangzw.vip/life", title: "🏠 生活" },
    { path: "https://yangzw.vip/about", title: "⚡ 关于" },
    { path: "https://yangzw.vip/contact", title: "📧 联系" },
  ],
  path: "https://yangzw.vip",
  title: "个人官网"
}
];
});

```

代码示例

dumi 在编写示例这件事上始终有着自己的理念：**开发者应像用户一样编写示例**。在 **md文件** 中编写超过30行的代码，其实对开发者来说很不友好，毕竟无法像真正的开发环境那样即时编写即时显现，再加上编码就像打开文本工具写文字一样无感，因此 **dumi** 提供一种新颖的示例接入方式，**外部示例**。

```
<code src="docs/app/constants/util/setting.js" />
```

md

这种外部示例使得开发者可像用户一样利用编辑器的 **Snippets**、**Stylelint**、**ESLint**、**Prettier** 等强大功能，再大的示例也能轻松维护。建议只要示例超过30行代码就使用上述方式解决。

模块引用

在编写示例时可能会引用到一些第三方模块，例如 `import React from "react"`，因为引用模块在 `node_module` 文件夹，这使得示例无法映射到文档中，需增加别名映射。

在 `.umirc.ts` 中指定 `chainWebpack`，设置别名映射。

```

import { dirname, resolve } from "path";
import { fileURLToPath } from "url";

const __filename = fileURLToPath(import.meta.url);

```

ts

```
const __dirname = dirname(__filename);

export default defineConfig({
  // ...
  chainWebpack(res) {
    res.resolve.alias.set("react", resolve(__dirname, "src/components"));
  }
});
```

这样就能完成一个体验不错的 文档站点 了，当然还有更多配置可参照[dumi配置列表](#) 自行定制。执行 `yarn run deploy` 打包 文档站点 。

打开 **FTP工具**，在 `/www/static` 目录中创建 `docs` 文件夹，该文件夹用于存放域名中所有文档资源。在 `docs` 文件夹中创建 `bruce` 文件夹，将打包好的 文档站点 的静态文件全部上传上去。



部署

通过第6章的方式在服务器中配置一个全新的 `doc.yangzw.vip` 二级域名，该域名用于托管所有 文档站点 。

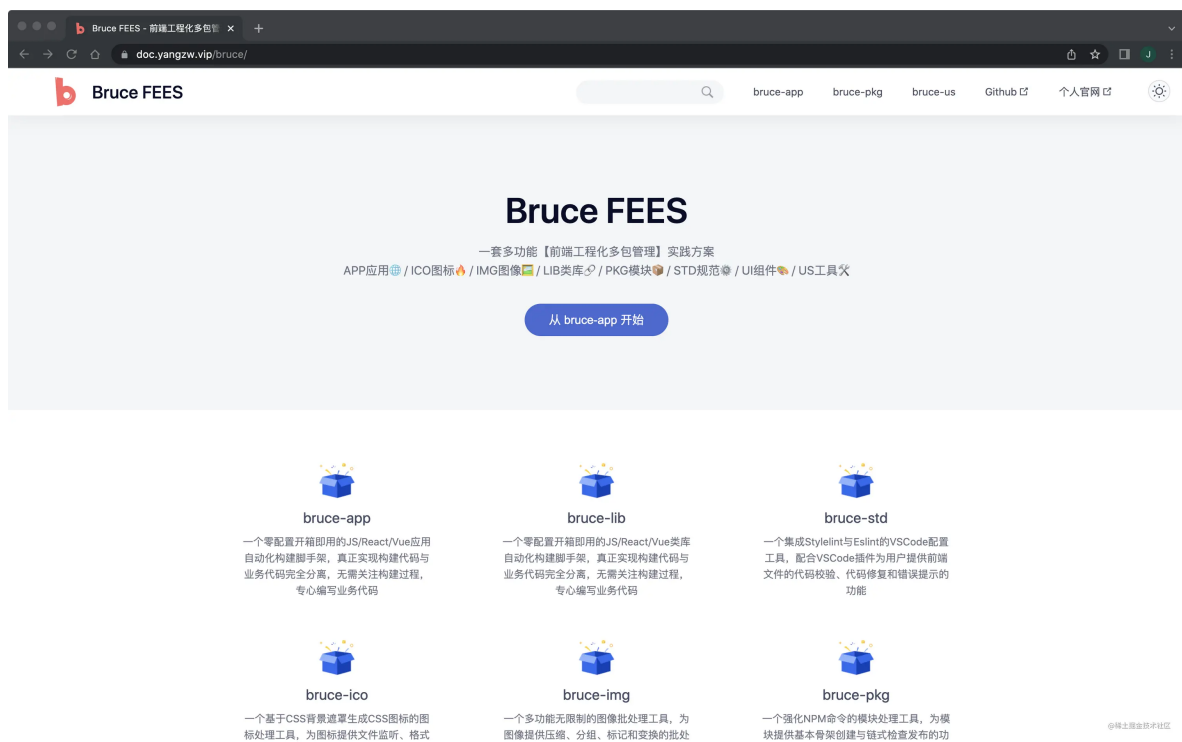
在 `/etc/nginx/conf.d` 目录中创建 `doc.yangzw.vip.conf` 文件，执行 `vim /etc/nginx/conf.d/doc.yangzw.vip.conf`，加入以下内容。

```
server {  
    listen 80;  
    server_name doc.yangzw.vip;  
    location / {  
        root /www/static/doc;  
        index index.html;  
        try_files $uri $uri/ /index.html;  
    }  
    location /bruce {  
        alias /www/static/doc/bruce;  
        index index.html;  
        try_files $uri $uri/ /index.html;  
    }  
}
```

执行 `certbot --nginx`，选择二级域名 `doc.yangzw.vip`，输出以下信息表示配置成功。

```
Successfully received certificate.  
Certificate is saved at: /etc/letsencrypt/live/doc.yangzw.vip/fullchain.pem  
Key is saved at:          /etc/letsencrypt/live/doc.yangzw.vip/privkey.pem  
This certificate expires on 2022-07-23.  
These files will be updated when the certificate renews.  
Certbot has set up a scheduled task to automatically renew this certificate in the background.  
  
Deploying certificate  
Successfully deployed certificate for doc.yangzw.vip to /etc/nginx/conf.d/doc.yangzw.vip  
Congratulations! You have successfully enabled HTTPS on https://doc.yangzw.vip
```

最后执行 `nginx -t` 验证 Nginx 配置，再执行 `nginx -s reload` 重启 Nginx 进程。在浏览器地址栏中输入 doc.yangzw.vip/bruce 就可正常访问自己的 Npm 私有仓库了。



总结

接着只需完善每份 **md文件** 就能生成漂亮美观的文档内容了。曾经一个完善的 **文档站点** 需从零到一手把手开发一遍，就像开发一个大项目那样引用很多工具链才能完成。

如今得益于各大前端框架源源不断的生态环境，造就了这么多生态衍生工具，实属感叹！当然可根据自己喜好选择适合自己的 **前端文档生成框架**，像 **vue** 技术栈的同学就选择 **vuepress** 吧，但不能享受那些 **dumi** 强大的动态生成功能了。

本章内容到此为止，希望能对你有所启发，欢迎你把自己的学习心得打到评论区！

☑ 示例项目：[fe-engineering](#)

☑ 正式项目：[bruce](#)

留言

输入评论（Enter换行，Ctrl + Enter发送）

发表评论

全部评论 (1)



橙寺布足   高级划水大师 @ 无业 3月前

非常棒 大佬辛苦了 晚上12点才发布上线 😊

 2  回复